

QuickLit: An LLM-Assisted System for Literature Retrieval, Summarization, and Exploration

Ziyue Gong 1005710740

Yuanrong Han 1010787409

December 8, 2025

Permissions

Group Member	Post Video	Post Final Report	Post Source Code
Ziyue Gong	Yes	Yes	Yes
Yuanrong Han	Yes	Yes	Yes

Word Count

1952 words (excluding references, tables, figures, and captions)

1 Introduction

The motivation for our project is to reduce the manual grind of finding and skimming papers—by automating retrieval and summarization with LLMs—so users can quickly see which papers matter and why. Thus, our goal is to develop an agentic system that automates the literature retrieval, summarization, and understanding workflow—specifically, the system (1) retrieves relevant papers given a research topic; (2) summaries extend beyond abstracts and conclusions; (3) provides an integrated chatbot for users to explore the literature interactively.

Given a research topic such as “Security Risk and Attacks in AI” or “Graph Retrieval-Augmented Generation for Large Language Models, the system automatically constructs structured search queries, retrieves arXiv papers, filters them using semantic similarity, and returns a ranked list with stable (`paper_id`, `title`, `pdf_url`) triples. On demand, QuickLit can then summarize individual papers at the section level and support interactive chat grounded in the retrieved corpus using a retrieval-augmented generation (RAG) pipeline.

2 Background

Elicit is a closely related system with similar goals—natural-language querying, ranked retrieval, conversational interaction, and summarization[1][2]. However, it offers stronger cross-paper synthesis (e.g., structured matrices, export tools, and auto-generated literature reviews). In comparison, QuickLit currently focuses on retrieval and section-level summarization rather than deeper synthesis.

An external study evaluating Elicit in AI-assisted literature workflows highlights repeatability, human-rated accuracy for one topic, and screening reliability [2]. QuickLit also measures accuracy, but via cosine similarity across multiple topics. Adding a repeatability check—such as the stability of top-k results—would better align with that framework.

Both systems share limitations: they lack access to paywalled content, and they can query only via Semantic Scholar and QuickLit via arXiv. While effective for rapid scoping, both still require human screening for final relevance, especially since Elicit can occasionally autocorrect queries (e.g., “ontology” → “oncology”).

A final note that we discovered Elicit only after completing QuickLit. The alignment in objectives and workflow retrospectively validates our system design and highlights the broader demand for AI-supported literature discovery and evaluation tools.

3 Data and Labels

3.1 System I/O Data

QuickLit ingests and emits the following:

- **Input (query):** a research topic.
- **Output (retrieval):** up to 30 arXiv papers per topic, each with title, URL, abstract, and an on-demand summary.
- **Input (follow-up questions to chatbot):** questions about the retrieved papers.
- **Output (chatbot answers):** responses that quote relevant parts of the retrieved papers.

3.2 Evaluation Data and Labels

See Section Metrics of Success.

4 Architecture and Software

4.1 High-Level System Architecture

QuickLit is deployed as a web application with three main components. Figure 1.

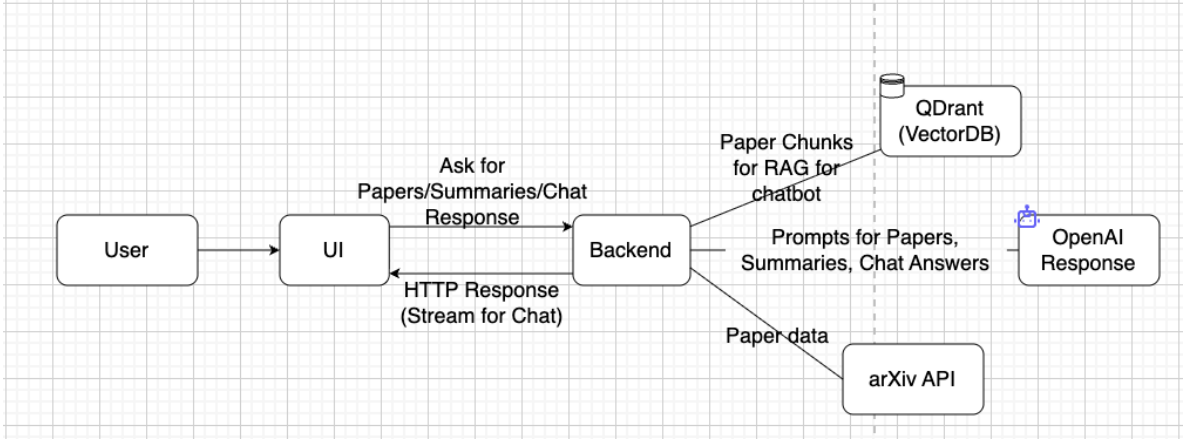


Figure 1: High-Level System Architecture

- **Frontend:** a web UI (in `quicklit-web/`) that allows users to enter topics, browse retrieved papers, request summaries, and chat with an assistant grounded in selected papers.
- **Backend API:** a FastAPI application that exposes endpoints for querying papers, building Qdrant collections, running RAG queries, streaming chat completions, and generating summaries.
- **Vector store:** a Qdrant instance used to store chunked paper text and enable semantic retrieval for the RAG chatbot.

The entire system is orchestrated with Docker Compose, with separate containers for the frontend, backend, and Qdrant. Environment variables configure the OpenAI API key and Qdrant host/port.

4.2 Retrieval Pipeline

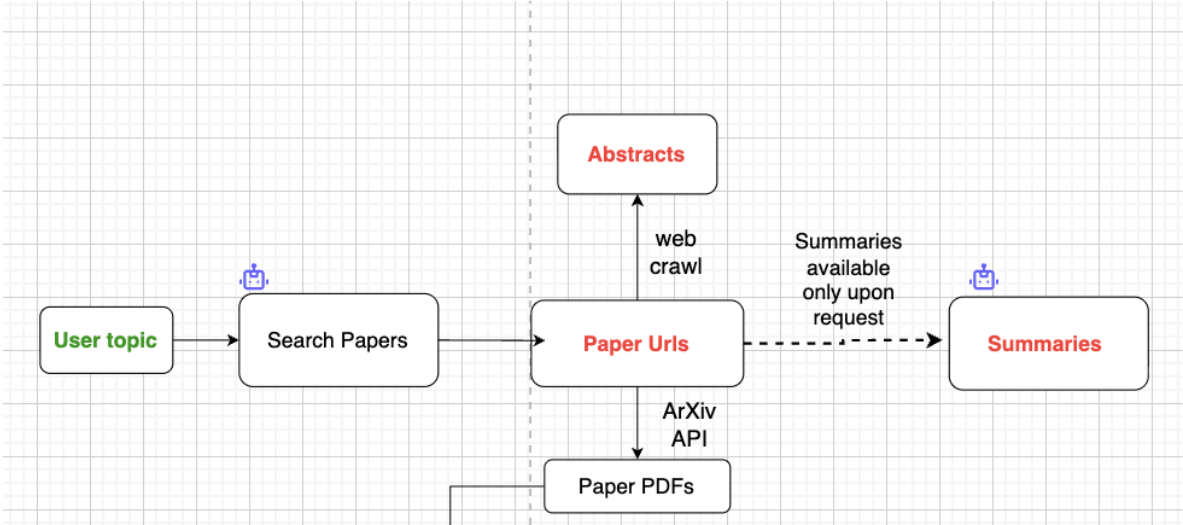


Figure 2: Paper Retrieval Pipeline

Given a topic, the retrieval pipeline follows the sequence illustrated in Figure 2:

1. A search agent (`gpt-5`) searches for papers by extracting core concepts, forming structured queries, and retrieving arXiv paper triples (`id`, `title`, `url`).
2. The system crawls abstracts using the URLs.

3. When requested by the user, the URL is passed to a summary-generation agent (`gpt-5`) to produce section-level summaries.
4. Full-text PDFs are fetched using the same URLs and ingested into the vector database for downstream chat.

4.3 RAG and Chat Pipeline

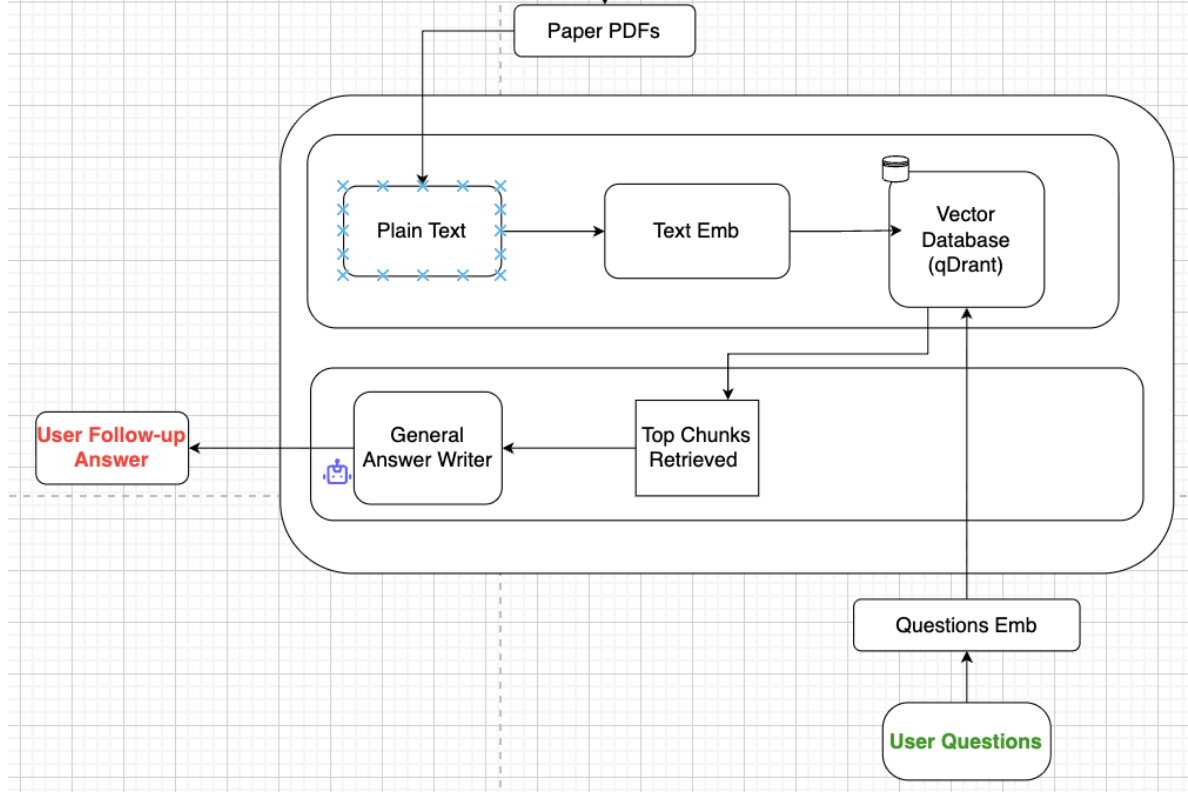


Figure 3: RAG and Chat Pipeline

Once papers are retrieved,

1. Fetch papers and convert to plain text.
2. Split and embed text, store in a Qdrant vector database along with chunk metadata.

Once a user inputs a question:

1. The question is embedded and used to query Qdrant for the top- k most relevant text chunks.
2. These retrieved chunks and the user's question are passed to a general answer-writing LLM (`gpt-5`).
3. The LLM synthesizes an answer and returns it to the user.

5 Metrics of Success

We report metrics for retrieval, summarization, RAG-based ranking, and chat quality.

5.1 Retrieval Metrics

Data. Gathered papers for 5 versions of prompts, for each of 5 evaluation topics: (1) Deep Reinforcement Learning for Data Processing and Analytics, (2) Security Risk and Attacks in AI, (3) Graph Retrieval-Augmented Generation for Large Language Models, (4) Machine Learning Approaches and Its Techniques, and (5) Edge Computing Systems and Tools.

Metrics.

- Mean and median cosine similarity between the topic embedding (using model: all-MiniLM-L6-v2) and each paper’s embedding (title and abstract).
- Precision of retrieved papers with cosine similarity ≥ 0.4 .
- Top-10 precision at threshold ≥ 0.4 .

Success Criterion. We refer to an external benchmark for all-MiniLM-L6-v2: In the QT (Question Target) corpus for clinical question answering, cosine similarity among highly relevant abstracts ranges from **0.59–0.90** (FQ1) and **0.55–0.99** (FQ2) [3].

5.2 Summarization Metrics

Data: Section-level summaries were generated for three arXiv papers (2504.00761, 2506.09159, 2510.09851). Each paper includes arXiv metadata and a parsed PDF URL.

Metrics: An LLM judge assigns 1–10 scores for:

- Section coverage: covers all first-level sections. Captures the core ideas of each section.
- Concision: each section is concise while maintaining clarity and completeness.
- Faithfulness: ideas mentioned in the summary have direct references in the original paper.
- Writing quality: grammar, clarity, coherence.
- Overall quality: weighted holistic score.

Success Criterion. Summaries scoring ≥ 8 in coverage and faithfulness; concision and writing quality are secondary.

5.3 Mean Reciprocal Rank for RAG Retrieval

We use Mean Reciprocal Rank (MRR) to quantify how well the RAG retriever surfaces the correct paper near the top of the ranked list. For a given query, let r be the rank (1 for top result, 2 for second, etc.) of the first retrieved chunk whose `paper.title` matches the true target paper; the reciprocal rank is then $RR = 1/r$, and it is 0 if the target never appears in the top- k . Across a set of queries Q , the mean reciprocal rank is

$$\text{MRR}@k = \frac{1}{|Q|} \sum_{q \in Q} \text{RR}(q).$$

The script `backend/eval/retrieve_eval/eval_mrr.py` evaluates this for three held-out arXiv papers. It downloads each paper, parses the PDF into text, chunks the content, and stores it in a dedicated Qdrant collection, then issues four natural-language queries per paper (asking for its summary, key contributions, methods, and problem statement). For each query, it calls `query_qdrant_openai` with `top_k = 10`, computes the reciprocal rank using the helper `reciprocal_rank`, and finally reports the overall MRR@10 along with per-query RR values. Running this script end-to-end thus provides an automated regression test of RAG retrieval quality.

Data Collected Three arXiv papers (2511.04213, 2511.01956, 2511.03070) were ingested as parsed chunks (abstract + PDF). For each paper, four query variants were asked: (1) “summary of {title}”, (2) “key contributions of {title}”, (3) “methods used in {title}”, and (4) “which problem {title} addresses”. Evaluation is under `backend/app/eval_mrr.py`.

5.4 Chat Metrics

Data User follow-up questions paired with grounded chatbot answers, scored by model-based evaluators.

Rubric (agentic judge). Weighted scoring over five dimensions:

- Grounding accuracy (0.35),
- Responsiveness (0.20),
- Evidence and citation quality (0.15),
- Reasoning quality (0.15),
- Communication and safety (0.15).

Metric. The final chat score is a weighted sum on a scale of 0-5.

6 Quantitative Results

6.1 Retrieval Results

We evaluate retrieval performance across five query pipelines—two semantic baselines and three arXiv-only variants—using mean cosine similarity as the primary metric. Table 1 reports the mean similarity scores for the most recent arXiv-only pipeline (v5 in `compare_scores.csv`) across the five evaluation topics.

Topic	Mean similarity (v5)
Deep RL for Data Processing and Analytics	0.45
Security Risk and Attacks in AI	0.40
Graph Retrieval-Augmented Generation	0.60
Machine Learning Approaches	0.25
Edge Computing Systems and Tools	0.52

Table 1: Mean topic–paper similarity for the latest arXiv-only pipeline (v5) across evaluation topics.

6.2 Summary Scores

For a subset of edge-computing papers, we evaluated section-level summaries using the LLM judge rubric. Table 2 shows the resulting scores aggregated in `combined_summary_scores.csv`.

Paper (arXiv ID)	Cov.	Conc.	Faith.	Write.	Overall
2504.00761	9	8	9	8	8.0
2506.09159	10	8	10	9	9.0
2510.09851	10	9	10	9	9.5

Table 2: LLM-judged summary scores (1–10) for three edge-computing papers.

These scores indicate that the system can produce summaries with near-perfect faithfulness and coverage, and overall scores between 8 and 9.5 without any task-specific supervision.

6.3 MRR for RAG Retrieval

For this small RAG sanity check, we ingested three papers on LLM feedback for statistics exams, AI-supported education, and observational benchmarks for LLMs, and issued four natural-language queries per paper (summarizing, listing key contributions, describing the methods, and identifying the problem addressed). For all 12 queries, the correct paper was retrieved at rank 1, yielding a reciprocal rank of 1.0 for every query and an overall $\text{MRR@10} = 1.000$. This confirms that, once the relevant papers are in the Qdrant collection and the query explicitly references the target title, the RAG retriever reliably surfaces the correct document at the top of the ranked list; more challenging paraphrased queries and larger candidate sets are left for future evaluation.

6.4 Chat Scores

We ran the agentic chat evaluator on seven saved conversations in `backend/eval/chat_eval/`, spanning single-paper question answering (e.g., SimClass; advantages and disadvantages of ChatGPT in education) and multi-paper synthesis of scoping reviews and policy analyses. Each conversation was scored on a 0–5 scale using the weighted rubric from Section *Metrics of Success*. Across runs, weighted scores ranged from **3.0** for early multi-paper summaries using smaller context chunks (with some hallucinated counts and missing citations) up to **5.0** for runs that reused the same prompts but evaluated chats built from larger chunk sizes. Typical strong conversations achieved weighted scores between **4.15** and **4.85**. The judge consistently flagged two main failure modes: incomplete coverage of all included papers and insufficient explicit evidence. Notably, when we switched to larger chunk sizes for the same tasks, hallucination flags disappeared, and weighted scores improved.

Scenario	Score	Main issues
Single-paper QA	4.70	evidence
Multi-paper overview	3.00	hallucination, coverage, other
Single-paper pros/cons	4.15	coverage
Multi-paper focus	4.85	coverage
Multi-paper focus (large chunk)	5.00	none
Single-paper adv./disadv.	4.65	coverage
Single-paper adv./disadv. (large chunk)	4.85	none

Table 3: Agentic judge chat scores across saved conversations.

7 Qualitative Results

For paper retrieval, as shown in Table 1, two topics: "Graph Retrieval-Augmented Generation" and "Edge Computing Systems and Tools" achieve mean similarities near or above **0.50** and also exhibit **top-10 precision of 1.0**. These scores fall within the QT reference range, indicating good retrieval. In contrast, broad or underspecified queries such as "Machine Learning Approaches" produce substantially lower similarity **0.25**. This mirrors the behavior observed in the QT corpus, where similarity decreases when queries become less specific or more heterogeneous.

On the summarization side, manual inspection of the MOSE paper shows that the generated section-by-section summaries correctly identify system components, experimental setups, and key results, such as improvements in migration latency and service availability, without inventing non-existent baselines or metrics. The LLM judge’s explanations corroborate this, highlighting that the summaries “cover all primary sections” and “preserve nuance” from both benefits and limitations sections.

For chat, example traces show that the agentic judge rewards answers that enumerate both advantages and disadvantages of LLMs in education, grounding each point in specific cited works (e.g., surveys on LLM usage in postgraduate education), and penalizes responses that omit key trade-offs or fail to reference the underlying literature.

8 Discussion and Learning

A key lesson learned was the importance of selecting evaluation metrics. Our initial paper retrieval metric—title matching against survey-derived citation lists—proved ineffective, not because the system retrieved poor papers, but because the gold lists did not comprehensively cover each topic. Later, we switched to metrics grounded in prior literature: embedding-based cosine similarity, which offered clearer interpretability and more stable signals.

We also observed practical limitations of the agentic system: it can be slow and non-deterministic, producing minor variations across runs. Browser-level caching helped mitigate this, and future work will explore lighter or more deterministic models to support reproducible evaluation.

Also, adding a small set of human-labeled summaries and chat ratings could help calibrate our metrics. We also aim to explore diversification objectives in retrieval, ensuring coverage across subtopics rather than ranking solely by text similarity.

9 Individual Contribution

Ziyue Gong

- **Paper Retrieval:** Designed and iterated the retrieval endpoint across five prompt versions (Scholar Semantic API $\rightarrow$ arXiv v3–v5); implemented evaluation for five topics across all versions and aggregated results into comparison tables.
- **Summarization:** Implemented the summary-generation endpoint; designed and ran an LLM-judge-based evaluation for three summaries.

Yuanrong Han

- **System Deployment:** Set up Docker-based containerization for the Qdrant vector store, backend API, and React frontend, and maintained the Compose configuration for local development.
- **Frontend Implementation:** Initialized the QuickLit web frontend and refactored the React components and layout to support topic search, paper browsing, and summary display.
- **RAG and Chat Pipeline:** Built the retrieval-augmented generation pipeline connecting retrieval, Qdrant, and the LLM, and implemented the chatbot backend and UI flow for grounded question answering over selected papers.

References

- [1] J. Y. Kung, “Elicit,” *Journal of the Canadian Health Libraries Association*, 2023.
- [2] N. Bernard, Y. Sagawa Jr, N. Bier, and Others, “Using artificial intelligence for systematic review: the example of elicit,” *BMC Medical Research Methodology*, 2025.
- [3] F. Ciampi, B. Giannini, G. Marzi, and A. Papa, “Artificial intelligence for systematic reviews: Evidence from the use of all-minilm-l6-v2 embeddings in clinical question answering,” *Information*, vol. 15, no. 2, p. 68, 2023.